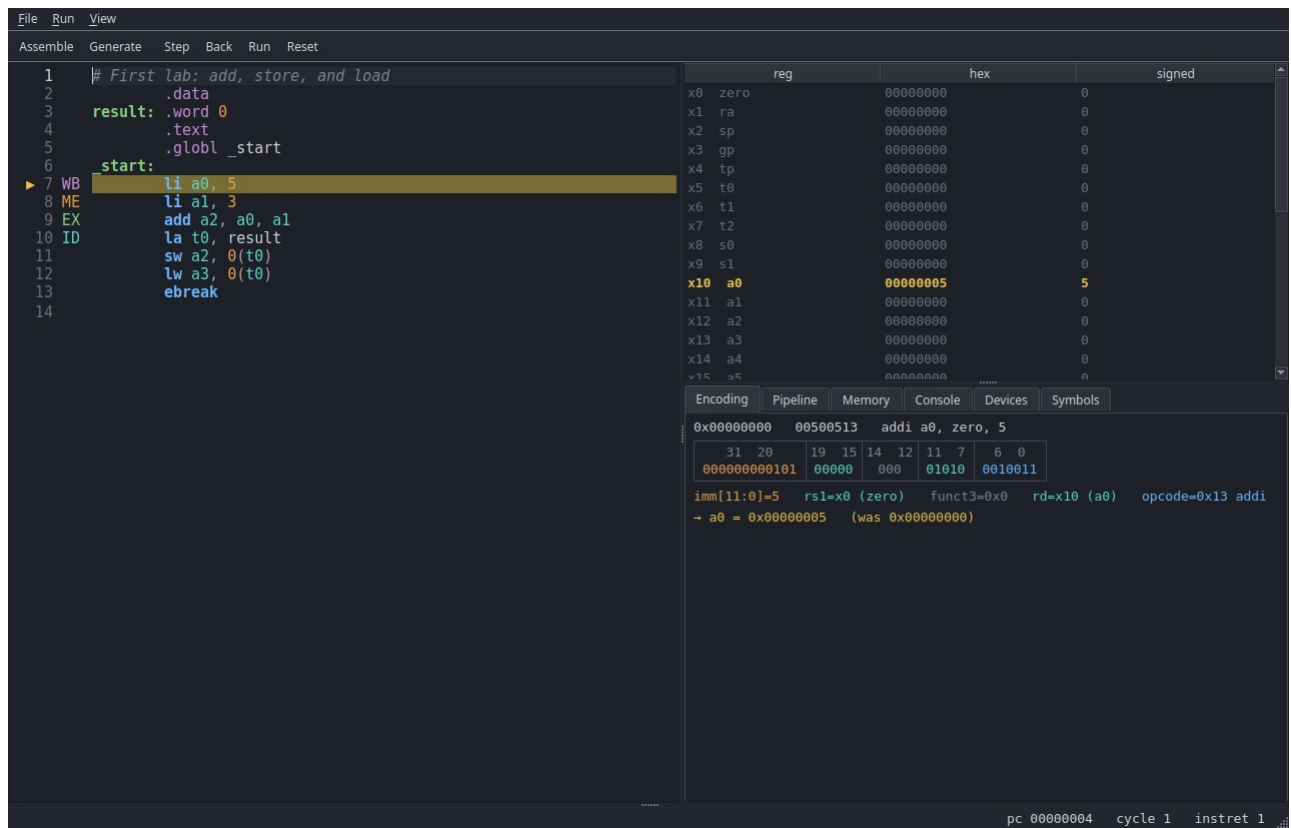


rv32 · Student usage guide

From your first instruction to a working memory image

A practical companion for assembly and computer architecture labs. Follow pages 2–5 for your first session; use the remaining pages as a reference during coursework.



The actual rv32 interface after the first instruction reaches write-back. Registers show a0 = 5; the Encoding tab explains the instruction.

Read	What you will do
2 · Set up	Open the application and organize your lab files.
3 · First program	Add two numbers, store the answer, and load it back.
4 · Read the window	Interpret registers, memory, symbols, and instruction encoding.
5 · Debug a loop	Use breakpoints, reverse stepping, and pipeline stages.
6 · Write and fix code	Use completion and diagnose common mistakes.
7 · Peripherals	Print to UART and work with switches and LEDs.
8 · Export	Generate correctly sized instruction and data images.
9 · Terminal tools	Run, trace, assemble, and debug without the GUI.
10 · Lab reference	Practice tasks, checks, and essential architecture rules.

Prepared 13 September 2026 for the inspected repository and local rv32 1.1.0 build. Screenshots were captured from the application. Course-specific submission rules and board settings come from your instructor.

2 · Set up your lab

Use the application supplied by your instructor when available. You do not need a separate RISC-V assembler or compiler to write and run assembly in rv32.

Windows

1. Extract the supplied `rv32-windows-x64.zip` into a folder you can write to.
2. Keep the entire extracted folder together, including the Qt DLLs and `platforms` folder.
3. Double-click `rv32-gui.exe`.

Linux

If rv32 is already installed, open it from the applications menu or run `rv32-gui`. If given an AppImage and the repository's installer, run:

```
bash /path/to/install.sh ~/Downloads/rv32-x86_64.AppImage
```

The installer installs under your user account, without `sudo`, and unpacks the AppImage to avoid a FUSE dependency. Use the actual paths to your downloaded files. A standalone AppImage can also run with `chmod +x rv32-x86_64.AppImage` followed by `./rv32-x86_64.AppImage` when FUSE support is available.

Using an existing source build

From the repository folder, launch `./build/debug/rv32-gui` (or the corresponding release build). Building the application is only necessary if your instructor asks for it; the repository's `docs/building.md` covers that workflow.

Keep each exercise together

```
lab01/  
  first_steps.s      # your editable assembly source  
  board.toml         # if supplied for this exercise  
  exports/           # generated memory files
```

This guide includes four ready-to-open files in its `examples` folder. Make a working copy before changing one. In the GUI, use **Ctrl+O** to open a file and **Ctrl+S** to save it. Use **File** → **Save As...** for a new copy.

Working routine: choose the board and memory sizes → edit → save → assemble → inspect/run → export. After changing code, press F5 again before expecting execution or export to use the new version.

3 · Run your first program

Open `examples/first_steps.s`, or create a new file with `Ctrl+N` and enter the program below. Save it as `first_steps.s`.

```
# First lab: add, store, and load
.data
result: .word 0
.text
.globl _start
_start:
    li a0, 5
    li a1, 3
    add a2, a0, a1
    la t0, result
    sw a2, 0(t0)
    lw a3, 0(t0)
    ebreak
```

1. Press **F5 · Assemble**. Resolve any errors before continuing. A note about `la` expanding into two instructions is expected.
2. Press **F8 · Step** five times from the newly assembled state. The pipeline must fill before the first instruction commits; `a0` then becomes 5.
3. Continue stepping. Watch `a1` become 3, then `a2` become 8. `la` takes two machine instructions even though it occupies one source line.
4. Select **Memory**. When `sw` commits, the word at data address 0 becomes `00000008`. When `lw` commits, `a3` becomes 8.
5. Press **F9 · Run** to finish. `ebreak` stops the program intentionally.

Check at the end	Expected value
<code>a0 / x10</code>	5
<code>a1 / x11</code>	3
<code>a2 / x12</code> and <code>a3 / x13</code>	8 (0x00000008)
<code>t0 / x5</code>	0: the data address of <code>result</code>
DMEM address 0	00000008

Try it: change 5 to 12, save, and press `F5` then `F9`. Both `a2` and `a3` should be 15. Pressing `Reset` alone does not assemble your edits.

4 · Read the machine state

The screenshot shows the RV32 simulator interface. On the left, the assembly code is displayed with line numbers 1 through 14. Line 13 is highlighted with a yellow background and labeled 'WB' (write-back). The code includes comments and instructions for setting up a program, including a data label 'result' and a text label '_start'. The main part of the code consists of several instructions: 'li a0, 5', 'li a1, 3', 'add a2, a0, a1', 'la t0, result', 'sw a2, 0(t0)', 'lw a3, 0(t0)', and 'ebreak'. On the right, the 'Registers' panel shows the state of 32 registers. The 'x10' register (a0) is highlighted in yellow and contains the value 5. Other registers like 'x11' (a1) contain 3, 'x12' (a2) contain 8, and 'x13' (a3) contain 8. Below the registers, the 'Memory' panel shows a table of memory addresses and their contents. The first row shows address 00000000 with a value of 00000008. The status bar at the bottom indicates the program is stopped at the 'ebreak' instruction, with PC 00000020, cycle 8, instret 8, and HALTED.

The first program after its load has committed. Address 0 in data memory contains 00000008.

Panel	How to use it
Registers	Match ABI names such as a0 with numbered names such as x10. Hex and signed decimal show the same 32 bits; FFFFFFFF is −1 in signed decimal. The last written register is highlighted.
Memory	Read a cell using row address + column offset. In row 00000010, column +4 is byte address 0x14. Each cell is one 32-bit word. You can edit values in hex.
Encoding	Inspect bitfields and the instruction's effect. It follows write-back while stepping or the source caret when you select a line. Compressed instructions also show their base instruction.
Symbols	Find the value and address space of labels. result belongs to Data; _start belongs to Text.
Console	Read bytes sent to the UART. Arithmetic alone produces no console output.
Devices	Inspect and operate attached inputs, outputs, and memory blocks.

Separate memories, shared-looking addresses

IMEM holds instructions; DMEM holds data. Both start at numeric address 0, but they are different storage. `lw` and `sw` access data memory or peripherals, never instruction memory. Use `la t0, result` for a data label, and `j` or `call` for a code label.

Word accesses need addresses divisible by 4; halfword accesses need addresses divisible by 2. The machine is little-endian. A word 0x12345678 occupies bytes 78, 56, 34, 12 at increasing addresses.

5 · Debug a loop

Open `examples/sum_loop.s` and press F5. The loop adds $1 + 2 + 3 + 4 + 5$ into `a0`.

1. Click the editor margin beside `add a0, a0, t0` to set a breakpoint. Use an executable line rather than a blank line.
2. Press F9 to reach the breakpoint. Inspect the accumulator and loop counter before advancing.
3. Use F8 to advance cycles until the `add` commits. Use F6 to undo cycles and watch the committed change reverse. F6 is disabled when no cycle history remains.
4. Click the same margin to remove the breakpoint, then F9 to finish. Expect `a0 = 15` and `t0 = 6`.
5. Press F4 to restart the loaded program. Reset keeps breakpoints; remove them if you want an uninterrupted run.

The screenshot shows the rv32 GUI with the assembly editor on the left, the register window on the right, and the pipeline tab selected at the bottom.

Assembly Editor:

```
1  # Sum 1 through 5
2  .text
3  .globl _start
4  _start:
5      li t0, 1
6      li t1, 6
7      li a0, 0
8  loop:
9      ID add a0, a0, t0
10     WB addi t0, t0, 1
11     ME blt t0, t1, loop
12 done:
13     ebreak
14
```

Register Window:

reg	hex	signed
x0 zero	00000000	0
x1 ra	00000000	0
x2 sp	00000000	0
x3 gp	00000000	0
x4 tp	00000000	0
x5 t0	00000002	2
x6 t1	00000006	6
x7 t2	00000000	0
x8 s0	00000000	0
x9 s1	00000000	0
x10 a0	00000001	1
x11 a1	00000000	0
x12 a2	00000000	0
x13 a3	00000000	0
x14 a4	00000000	0
x15 a5	00000000	0

Pipeline Tab:

cycle 9

	IF	ID	EX	MEM	WB
addi	0010	000c	--	0014	0010

the register file is written here ↑

IF fetch · ID decode · EX ALU · MEM data memory · WB register write
branch target computed in EX · guessed taken · checked at WB

cycle

	1	2	3	4	5	6	7	8	9
addi	0000	IF	ID	EX	ME	WB			
addi	0004		IF	ID	EX	ME	WB		
addi	0008			IF	ID	EX	ME	WB	
add	000c				IF	ID	EX	ME	WB
addi	0010					IF	ID	EX	ME
blt	0014						IF	ID	EX
ebreak	0018							IF	
add	000c								IF
addi	0010								

pc 00000014 cycle 5 instret 5

A loop in flight. The Pipeline tab shows stage occupancy and an instruction-by-cycle history.

IF fetches, **ID** decodes, **EX** represents execution and target calculation, **MEM** represents data access, and **WB** commits the instruction. The yellow source line marks write-back. Resolved branches are green when taken and red when they fall through.

Interpret timing carefully: this is a teaching visualization, with no hazards, stalls, or forwarding. The engine commits a whole instruction at WB; branch outcomes also resolve there. The panel's cycle count differs from the program-visible counters. Do not use it to measure FPGA cycle timing or CPI.

To step over a function in the GUI, place a breakpoint on the executable line after its call and run to it. F9 again interrupts a running program.

6 · Write code and fix mistakes

Type two letters of a mnemonic, or press **Ctrl+Space**, to open suggestions. **Tab** / **Shift+Tab** cycle choices; **Enter** accepts; **Esc** closes the suggestions. Grey operand hints are placeholders, not saved text. Enter registers, commas, and values yourself.

Symptom	What to check or change
Undefined symbol	Compare spelling and case with the label definition. Click the diagnostic to jump to its location.
Immediate out of range	addi accepts -2048 through 2047. Use li to load a larger constant, then a register arithmetic instruction where appropriate.
A call targets a data symbol	Put executable code in .text. Use la for the address of an object in .data.
Misaligned load/store	For lw/sw, check that the effective address is a multiple of 4. Avoid placing a word immediately after an unaligned byte or string without padding/alignment.
Red overflow markings	The program exceeds IMEM. Increase the configured memory if your lab permits, or reduce the program; reassemble. Check DMEM overflow messages too.
Illegal instruction or fetch fault	Check the stopping PC, jump targets, and whether execution fell beyond your code. End a simple exercise with ebreak.
Peripheral access fault	Verify that the board file attaches a device at the address your program accesses.
No register changes after a step	F8 advances one pipeline cycle. Wait for WB; also remember that writes to x0 are discarded.
Old behavior after editing	Save, then press F5 successfully. A failed assembly leaves the previously loaded program intact.
An endless loop	Interrupt with F9, set a breakpoint at the branch, and inspect its inputs. Confirm the counter advances toward the exit condition.
No console text	Open Console and verify a store to the UART transmit address. This tool does not provide another simulator's print ecall services.

A repeatable debugging method

1. Write down the expected register or memory value at a particular line.
2. Break before the region that produces it.
3. Step until the first difference appears, inspecting operands and addresses.
4. Use Back to revisit that change, fix the source, then save and assemble again.

Practice: in a copy of the loop, change `loop` to `lop` in the branch. Assemble, follow the diagnostic, correct it, and reassemble.

7 · Console and peripherals

Open `examples/uart.s`, assemble, run, and select Console. It writes ASCII 65 followed by ASCII 10, so the output is **A** followed by a newline.

```
# Print A and a newline on the default machine
.text
.globl _start
_start:
    li t0, 0xffff0000
    li a0, 65
    sb a0, 0(t0)
    li a0, 10
    sb a0, 0(t0)
    ebreak
```

Default address	Purpose
0xFFFF0000	UART: store a byte at +0 to transmit; status at +4.
0xFFFF0010	LED output register.
0xFFFF0020	Switch input register.
0xFFFF0030	mtime timer value.
0xFFFF0040	mtimecmp timer deadline.
0xFFFF0050 / 0xFFFF0054	Software / external interrupt request registers.

Try a board configuration

1. Choose **File** → **Load peripherals...** and open the included `examples/board.toml`. This replaces the attached devices and specifies memory sizes.
2. Open `switches_leds.s` and assemble. In Devices, inspect the switches; the supplied board starts them at binary 1010 (decimal 10).
3. Run. Expect `a0 = 10` and the corresponding LED value. The program reads once and stops.
4. Change the switches, reset, and run again to sample the new value. Switch inputs survive reset.
5. Use **File** → **Save peripherals...** to save a configuration you need to reproduce later.

Other devices include buttons, numeric inputs, displays, RAM/ROM blocks, custom registers, and a multiplier function device. Use the configuration supplied for the assignment; addresses may differ from the defaults. Select a RAM/ROM block for its own Load .mem and Export .mem controls.

For interrupt labs, start with the repository's `examples/timer.s`. Install a handler and enable the relevant interrupt before expecting it to run. Timer and instruction counters are not wall-clock or FPGA timing measurements.

8 · Export files for your hardware lab

An assembly source file is editable code. A .mem file is a memory image. Keep your .s file even after exporting.

1. Load the assignment's board configuration and set the required IMEM/DMEM sizes in the Memory tab. Save your source and press F5. Resolve errors and overflow markings.
2. Choose **Generate** on the toolbar, press **Ctrl+E**, or use **File** → **Generate .mem....**
3. Choose the destination folder and base name. Select imem and dmem as needed. With base name lab01, the outputs are lab01_imem.mem and lab01_dmem.mem. The default base name is rv32.
4. Use disassembly comments only if your receiving workflow accepts them. For a plain image, leave annotation unchecked.
5. Press Export. Match each displayed depth to the hardware memory: 16,384 bytes = 4,096 words, each 32 bits wide.

Which export do you need?

Action	Data written
Generate / Ctrl+E	The most recently assembled program and its initial .data contents, padded to the configured memory depths.
Memory → Export .mem...	The current DMEM contents, including changes made during execution or manual editing.
Devices → selected block → Export .mem...	That RAM/ROM block's current image, padded to the block's own size.

For the first example, Generate writes `result = 0` because that is the source initializer. After running, exporting the Memory tab captures `result = 8`. Generate does not automatically assemble later edits.

Read the file correctly

Each line contains one 32-bit hexadecimal word. Counting the first line as index 0, line index *n* corresponds to byte address $4 \times n$. Compressed instructions are packed into this word stream; one file line is not necessarily one instruction.

```
reg [31:0] imem [0:4095];
initial $readmemh("lab01_imem.mem", imem);
```

This is a Verilog simulation initialization example. Use the import/initialization procedure specified for your FPGA IP and lab. In a 32-bit Block Memory Generator, a 16 KB image corresponds to Write Depth 4096. Confirm the required file format for that flow; changing a filename extension does not convert formats.

CLI difference: rv32 asm exports only used memory by default. Include `--pad` for full-depth images, and the same `--devices board.toml` configuration when using custom memory sizes.

9 · Use the terminal tools

Run these commands from your working exercise folder. Replace rv32 with the path to the executable if it is not on PATH. On Windows PowerShell, use `.\rv32.exe` when it is in the current directory.

```
rv32 run first_steps.s
rv32 run first_steps.s --trace
rv32 run sum_loop.s --limit 1000
rv32 asm first_steps.s --pad -o lab01_imem.mem --dmem lab01_dmem.mem
rv32 dis lab01_imem.mem
rv32 run switches_leds.s --devices board.toml
rv32 asm first_steps.s --pad --devices board.toml
rv32 dbg sum_loop.s
```

The trace lists machine instructions and their effects, so a pseudo-instruction may produce multiple lines. The default run limit is 1,000,000 instructions. A limit stop means execution has not necessarily completed. A padded IMEM disassembly can include unused zero-filled memory after the program.

An interactive debugging session

```
(rv32) break done
(rv32) run
(rv32) info registers
(rv32) print a0
(rv32) backi
(rv32) si
(rv32) continue
(rv32) quit
```

Command	Meaning
step / s	One source line, stepping into calls.
stepi / si	One machine instruction.
next / n	One source line, completing calls.
finish	Continue until the current function returns.
back / backi	Undo one source line / machine instruction.
run / continue	Reset and run from entry / resume current state.
break done	Set a breakpoint on a code label.
watch result	Stop when a data word changes (in a program defining result).
x/4xw 0	Read four words of memory in hexadecimal from address 0.
help	List commands and their usage.

Stepping units differ: GUI F8/F6 use pipeline cycles; CLI step/back use source lines; CLI si/backi use machine instructions. An empty debugger command repeats the last command.

10 · Keep beside you during a lab

Key	Action
Ctrl+N / Ctrl+O / Ctrl+S	New / open / save source.
F5	Assemble and load the current editor contents.
F8 / F6	Advance / undo one pipeline cycle.
F9	Run; press again to interrupt.
F4	Reset the loaded program; keep breakpoints.
Ctrl+E	Export the assembled memory images.
Ctrl+Space	Open instruction suggestions.
Ctrl+Shift+P	Toggle pipeline markings in the editor.
Ctrl+Shift+B	Toggle resolved-branch colors.

Practice checkpoints

Task	Expected observation
first_steps: replace 5 with 12	a2 = a3 = 15; result at DMEM 0 is 0000000f.
sum_loop: change the bound from 6 to 11	Adds 1 through 10; a0 = 55 (0x37).
uart: replace 65 with 66	Console shows B and a newline.
switches_leds: change switch inputs, reset, run	a0 and LEDs reflect the newly sampled input.
Compare addi a0, a0, 1 with c.addi a0, 1	Same arithmetic effect, 32-bit versus 16-bit encoding; use separate copies and reassemble.

Before recording or handing in results

- Save the source and assemble it successfully.
- Run from a known initial state using the intended board and inputs.
- Check the requested registers, memory addresses, and output against your prediction.
- Record the input values, board configuration, expected result, and observed result.
- When asked for hardware images, export with the required sizes and inspect the filenames. Follow your instructor's actual submission requirements.

Architecture facts to remember

This tool supports RV32 integer instructions, multiply/divide, compressed instructions, machine CSRs, and fence.i. It has separate instruction/data memory and machine mode only. It does not load ELF programs or provide floating-point instructions. x0 always reads as zero. li, la, and call are pseudo-instructions. Compression is explicit: ordinary instructions do not automatically become c.* instructions. ecall raises an exception; use the provided UART method for simple output.

Further reading in the repository: docs/keybindings.md, docs/memory-map.md, docs/isa-support.md, and docs/building.md. This guide was checked against the GUI and command implementations; the included validation log records the executed examples.